GAIAKTA

The Earth, Made Visible

Processing Pipeline v1.1

Production-Hardened Blueprint

Raw Video → Gaussian Splat → Cesium Globe

v1.0 → v1.1 Changes:

★ Upload checksum + duplicate detection • ★ Worker heartbeat + lease model
★ SAM2 confirmation TTL + abandoned state • ★ Idempotent finalize sequence
◈ Asset lineage stub (future) • ◈ Cost/billing stub (future)

March 2026 • Confidential

1. What Changed: v1.0 → v1.1

Four production-hardening items were added based on system design review. Two future stubs were also scaffolded — not implemented, but adapter hooks are in place so integration later requires zero pipeline restructuring.

Item	Type	Stage	Status
`Upload checksum + duplicate detection`	Hardening	Ingest (new)	★ Implemented
`Worker heartbeat + lease model`	Hardening	All GPU stages	★ Implemented
`SAM2 confirmation TTL + abandoned state`	Hardening	Stage 2	★ Implemented
`Idempotent finalize sequence`	Hardening	Stage 9	★ Implemented
`Asset lineage explorer`	Future stub	All stages	◈ Hook only
`Cost dashboards + billing awareness`	Future stub	Ingest + GPU	◈ Hook only

2. Full Pipeline Flow (v1.1)

★ = new in v1.1 ◈ = future stub hook, no logic yet

Ingest Upload received → R2 multipart complete
★ **NEW** ★ NEW — checksum (SHA-256), duplicate detection, quota check, abuse gate
⚠ **CHECKPOINT** CHECKPOINT — upload integrity verified
Stage 0 Pre-flight validation → preflight.py — codec, duration, resolution, corruption
Stage 1 FFmpeg frame extraction → 4K, 2–3 FPS, adaptive sampling, near-duplicate removal
Stage 2 SAM2 subject segmentation → segment.py — keyframes + confidence score + mask propagation
★ **NEW** ★ NEW — TTL 24h starts. Reminder at 12h via OneSignal
⏸ **USER ACTION** User reviews 2D multi-frame preview — confirms or reselects subject
★ **NEW** ★ NEW — timeout → abandoned_confirmation state → graceful cleanup
Stage 3 COLMAP sparse reconstruction → runs on masked frames only
⚠ **CHECKPOINT** CHECKPOINT — point count, reprojection distribution, pose spread, track length, registered ratio ≥ 60%
Stage 4 Downscale 4K → 1080p → after COLMAP, validates count/aspect/pose alignment
Stage 5 splatfacto training → --downscale-factor 1
★ **NEW** ★ NEW — worker heartbeat every 60s. Stale → reclaim + retry. NaN/plateau → safe config retry
◈ **STUB** ◈ STUB — asset lineage: log model version + config hash (no-op at MVP)
Stage 6 Splat export → ns-export gaussian-splat → raw splat.ply + manifest
Stage 7 Post-process cleanup → postprocess.py — opacity filter + pre/post gaussian count diff
Stage 8 Quality scoring → check_quality.py — composite score: loss + reprojection + holdout render
◈ **STUB** ◈ STUB — cost tracking: log GPU seconds + R2 bytes (no-op at MVP)
★ **NEW** ★ NEW — Idempotent finalize: upload → verify checksum → write DB → mark complete → notify → delete raw
⚠ **CHECKPOINT** CHECKPOINT — CDN URL live, metadata valid, preview renderable
Done Splat live on Cesium globe — user notified via OneSignal

3. The Four Hardening Items

3.1 — Upload Checksum + Duplicate Detection

★ NEW IN v1.1 — Hardening Addition

- SHA-256 hash computed server-side immediately after multipart upload completes.
- Hash stored in scan_jobs.file_hash column in Supabase.
- Before creating a new job, query existing jobs for same file_hash — block duplicates with clear message to user.
- Quota check: enforce per-user upload size limit (configurable, default 10GB/day at MVP).
- If multipart upload is incomplete (not all parts received), reject with specific error before job enters queue.
- Abuse gate: rate-limit job creation per user_id — max 10 jobs/day at MVP.

Why this matters: without a checksum, a corrupt 3GB upload that passes pre-flight wastes 40+ minutes of RunPod GPU time. Duplicate detection prevents a user accidentally re-uploading the same video and paying twice for identical processing.

3.2 — Worker Heartbeat + Lease Model

★ NEW IN v1.1 — Hardening Addition

- When a RunPod worker claims a job, it writes a lease: scan_jobs.worker_id + scan_jobs.lease_expires_at = now() + 5 minutes.
- Worker pings Supabase every 60 seconds: UPDATE scan_jobs SET lease_expires_at = now() + 5min WHERE id = job_id.
- Orchestrator (Supabase Edge Function on cron) checks every 2 minutes for expired leases — any job where lease_expires_at < now() and status not in (complete, failed).
- Stale job → status reset to queued, worker_id cleared, retry_count incremented.
- After 3 failed retries → status = dead_lettered. Job goes to DLQ. User notified.
- Each stage transition resets the lease timer so long stages (COLMAP, training) do not false-expire.

Why this matters: RunPod containers crash. WSL2 OOMs. Network drops. Without a heartbeat, crashed jobs stay stuck in 'training' state forever with no recovery path. The lease model is the difference between a pipeline that needs babysitting and one that heals itself.

3.3 — SAM2 Confirmation TTL + Abandoned State

★ NEW IN v1.1 — Hardening Addition

- When job enters awaiting_confirmation, write confirmation_expires_at = now() + 24h to scan_jobs.

- At 12h: OneSignal push reminder — 'Your GAIAKTA scan is waiting for your review.'
- At 24h: Supabase Edge Function cron moves job to abandoned_confirmation state.
- abandoned_confirmation triggers cleanup: delete extracted frames from R2, release RunPod reservation, notify user the scan expired.
- User can re-upload to start a new job — previous job cleanly tombstoned.
- Multi-frame preview: show user 3 keyframes (beginning / middle / end of scan) not just one — reduces wrong-subject confirmation.
- Mask confidence score stored: if SAM2 confidence < threshold, flag preview with warning 'Low confidence — please verify subject carefully.'

Why this matters: without a TTL, a user who uploads and never confirms leaves the pipeline in a permanent pause state. At scale this becomes a resource leak — frames stored in R2 indefinitely, job queue polluted with zombie entries.

3.4 — Idempotent Finalize Sequence

★ NEW IN v1.1 — Hardening Addition

- Finalize is now a distinct pipeline state: finalizing (separate from quality_check and complete).
- Ordered steps, each idempotent (safe to re-run if interrupted):
- 1. Upload clean splat.ply to R2 → store upload_etag in scan_jobs
- 2. Verify: re-fetch splat from R2, compare SHA-256 to local hash → confirm upload integrity
- 3. Write scan record to Supabase scans table → mark scan as pending_publish
- 4. Validate CDN URL is live and reachable
- 5. Update scan_jobs.status = complete
- 6. Send OneSignal notification to user
- 7. Delete raw video from R2
- If process crashes at any step, resume from finalizing — each step checks if already done before re-executing.
- Raw video deletion is the LAST step — never delete source until all upstream steps confirmed.

Why this matters: if finalize crashes between 'write DB' and 'delete raw video' and you retry naively, you re-upload a duplicate splat and double-notify the user. Idempotent steps mean retrying is always safe — each step is a no-op if already complete.

4. Future Stubs (Hooks Only — No Logic at MVP)

These two items are not implemented at MVP. However, adapter hooks are scaffolded in pipeline.py so that adding them later requires no restructuring of the pipeline — just filling in the stub functions.

4.1 — Asset Lineage Explorer

◈ **FUTURE STUB — Asset Lineage Explorer**

Not implemented at MVP. Adapter hooks are in place for zero-rework integration later.

- Purpose: track every artifact produced per scan — raw video, masked frames, sparse point cloud, trained checkpoint, raw splat, clean splat.
- At MVP: pipeline.py calls log_lineage(stage, artifact_path, metadata) — function is a no-op (just returns True).
- Future: log_lineage writes to a scan_artifacts table in Supabase with stage, artifact type, R2 path, size, hash, timestamp.
- Enables: debugging bad scans, reproducibility, re-running individual stages without full pipeline re-run.
- Integration cost when ready: fill in log_lineage() — zero pipeline restructuring.

4.2 — Cost Dashboards + Billing Awareness

◈ **FUTURE STUB — Cost Dashboards + Billing Awareness**

Not implemented at MVP. Adapter hooks are in place for zero-rework integration later.

- Purpose: track per-scan GPU seconds, R2 storage bytes, egress, and RunPod spend.
- At MVP: pipeline.py calls log_cost(stage, metric, value) — function is a no-op (just returns True).
- Future: log_cost writes to a scan_costs table. Dashboard aggregates by user, date, stage.
- Per-user quota enforcement and billing integration slot into the same hook.
- Integration cost when ready: fill in log_cost() — zero pipeline restructuring.

5. State Machine v1.1

New states added in v1.1 are marked ★. All state transitions written to scan_jobs in Supabase in real time.

`uploaded`	Video received in R2, job created in scan_jobs
`ingesting`	★ NEW — checksum computed, duplicate check, quota check
`validating`	Pre-flight checks: codec, duration, resolution, integrity
`extracting_frames`	FFmpeg 4K frame extraction, adaptive sampling, dedup
`segmenting`	SAM2 keyframe analysis + mask confidence scoring
`awaiting_confirmation`	★ NEW — TTL 24h, reminder at 12h, timeout → abandoned_confirmation
`abandoned_confirmation`	★ NEW — user never confirmed, job cleaned up gracefully
`confirmed`	User approved subject mask, pipeline resumes
`reconstructing`	COLMAP sparse reconstruction on masked frames
`training`	★ NEW — splatfacto + heartbeat ping every 60s
`exporting`	ns-export gaussian-splat → raw splat.ply
`postprocessing`	Opacity filter, pre/post gaussian count diff
`quality_check`	Composite score: loss + reprojection + holdout render
`finalizing`	★ NEW — upload → verify → DB → notify → delete (idempotent)
`complete`	Splat live on Cesium globe
`failed`	Job failed — user notified with specific reason
`dead_lettered`	★ NEW — exhausted retries, moved to DLQ for manual review

6. Scripts to Build

Working commands from the Firefighter Memorial run (FFmpeg, COLMAP, splatfacto, ns-export) require no rewriting. The following scripts are new builds:

Script	Stage	What It Does	New in v1.1
`ingest.py`	Ingest	SHA-256 checksum, duplicate check, quota gate, abuse rate-limit	★ Yes
`preflight.py`	Stage 0	Codec, duration, resolution, corruption check via ffprobe	No
`segment.py`	Stage 2	SAM2 keyframe segmentation, confidence score, mask propagation, TTL trigger	★ Partial
`check_colmap.py`	Stage 3	Multi-metric geometry gate: point count, reprojection dist, pose spread, track length, ratio	★ Expanded
`postprocess.py`	Stage 7	Opacity filter + pre/post gaussian count diff check	No
`check_quality.py`	Stage 8	Composite score: training loss + reprojection + holdout render fidelity	★ Expanded
`finalize.py`	Stage 9	Idempotent 7-step finalize: upload → verify → DB → CDN check → complete → notify → delete	★ Yes
`heartbeat.py`	All GPU	Worker lease management: claim, ping, stale detection, retry, DLQ	★ Yes
`pipeline.py`	All	Master orchestrator + stub hooks for lineage + cost logging	★ Updated

7. Supabase Schema Additions (v1.1)

New columns and tables required to support the four hardening items:

scan_jobs table — new columns

Column	Type	Purpose
`file_hash`	TEXT	SHA-256 of raw video — duplicate detection key
`worker_id`	TEXT	RunPod container ID that holds current lease
`lease_expires_at`	TIMESTAMPTZ	Heartbeat deadline — orchestrator reclaims if past
`retry_count`	INT DEFAULT 0	Number of times job has been retried
`confirmation_expires_at`	TIMESTAMPTZ	SAM2 confirmation TTL — 24h from segmenting
`upload_etag`	TEXT	R2 ETag after splat upload — idempotent verify step
`mask_confidence`	FLOAT	SAM2 confidence score — flagged if below threshold
`quality_score`	TEXT	Good / Fair / Poor — from check_quality.py
`quality_detail`	JSONB	Raw metrics: loss, reprojection_error, holdout_fidelity
`finalize_step`	INT DEFAULT 0	Which idempotent finalize step completed last (1-7)

scan_artifacts table — lineage stub (schema only, no writes at MVP)

Column	Type	Purpose
`id`	UUID PRIMARY KEY	Artifact record ID
`scan_id`	UUID REFERENCES scans	Parent scan
`stage`	TEXT	Pipeline stage that produced artifact
`artifact_type`	TEXT	raw_video / frames / sparse / checkpoint / splat_raw / splat_clean
`r2_path`	TEXT	R2 storage path
`file_hash`	TEXT	SHA-256 of artifact
`size_bytes`	BIGINT	File size

`created_at`	TIMESTAMPTZ	When artifact was written

scan_costs table — billing stub (schema only, no writes at MVP)

Column	Type	Purpose
`id`	UUID PRIMARY KEY	Cost record ID
`scan_id`	UUID REFERENCES scans	Parent scan
`stage`	TEXT	Pipeline stage
`metric`	TEXT	gpu_seconds / r2_bytes / runpod_cost_usd
`value`	FLOAT	Metric value
`recorded_at`	TIMESTAMPTZ	When cost was logged

8. Build Order

Build and test each script standalone on your laptop before touching Docker. The order below respects dependencies — each script only needs tools already verified working.

Order	Script	Dependency	Test Method
1	ingest.py	hashlib (stdlib), Supabase client	Feed a known video, check hash in Supabase, re-feed same video → duplicate blocked
2	heartbeat.py	Supabase client	Start fake job, run heartbeat, kill process, verify orchestrator reclaims after 5min
3	preflight.py	ffprobe	Feed good video → pass. Feed short/corrupt video → specific error code
4	check_colmap.py	COLMAP output folder	Point at Firefighter Memorial COLMAP output → verify metrics parse correctly
5	postprocess.py	plyfile, numpy	Load Firefighter Memorial splat.ply → filter → verify gaussian count reduced, file valid
6	check_quality.py	nerfstudio log + COLMAP log	Parse Firefighter Memorial logs → verify Good/Fair/Poor score assigned
7	finalize.py	Supabase, R2 boto3, requests	Dry-run all 7 steps against test scan. Kill mid-way, re-run → verify idempotent
8	segment.py	SAM2 installed	Run on 5 Firefighter Memorial frames → verify mask + confidence score output
9	pipeline.py	All above	End-to-end run on Firefighter Memorial video → verify all state transitions in Supabase
10	Dockerfile	All scripts passing	Build image → run against raw video → verify splat.ply produced in R2

9. Prompts

I'm building GAIAKTA, a Gaussian Splat pipeline. Create a Python script at /mnt/b/projects/GAIAKTA/pipeline/ingest.py

Requirements:

- Python 3.12, uses only stdlib + supabase-py client

- Takes args: --video-path (local path to raw video) --user-id (UUID string) --scan-id (UUID string)

- Computes SHA-256 hash of the video file in 8MB chunks (file can be 3GB+, do not load into memory)

- Queries Supabase scan_jobs table for any existing row with same file_hash — if found, exit with code 2 and print "DUPLICATE: scan_id={existing_id}"

- Checks total upload size for user today (sum of file_size_bytes in scan_jobs where user_id=X and created_at > today midnight UTC) — if > 10GB, exit code 3 and print "QUOTA_EXCEEDED"

- Checks job count for user today — if > 10, exit code 4 and print "RATE_LIMITED"

- If all checks pass: UPDATE scan_jobs SET file_hash=X, file_size_bytes=Y, status='validating' WHERE id=scan_id

- Exit code 0 on success, print "INGEST_OK hash={hash}"

- Supabase URL and service key read from env vars SUPABASE_URL and SUPABASE_SERVICE_KEY

- Full error handling, all exceptions caught and printed with exit code 1

- Include a log_lineage() stub function that accepts (scan_id, stage, artifact_path, metadata_dict) and just returns True — comment says "TODO: write to scan_artifacts table when lineage explorer is implemented"

- Include a log_cost() stub function that accepts (scan_id, stage, metric, value) and just returns True — comment says "TODO: write to scan_costs table when billing is implemented"

```
cd /mnt/b/projects/GAIAKTA

source venv/bin/activate

python pipeline/ingest.py --video-path /path/to/test.mp4 --user-id test-user --scan-id test-scan-id
```

SCRIPT 2 — `heartbeat.py`

Paste this into Claude CLI:

```
I'm building GAIAKTA. Create a Python script at /mnt/b/projects/GAIAKTA/pipeline/heartbeat.py
```

Requirements:

- Python 3.12, uses supabase-py

- Two modes via --mode arg: "worker" and "orchestrator"

WORKER MODE (--mode worker --scan-id X --worker-id Y):

- On startup: UPDATE scan_jobs SET worker_id=Y, lease_expires_at=now()+5min WHERE id=X

- Then loop forever: every 60 seconds UPDATE scan_jobs SET lease_expires_at=now()+5min WHERE id=X AND worker_id=Y

- If update affects 0 rows (lease was reclaimed by orchestrator), exit code 2 — print "LEASE_LOST: another worker claimed this job"

- Designed to run as a background thread inside pipeline.py — expose start_heartbeat(scan_id, worker_id) and stop_heartbeat() functions using threading.Event

- On stop_heartbeat(), clear worker_id and lease_expires_at

ORCHESTRATOR MODE (--mode orchestrator):

- Query scan_jobs where lease_expires_at < now() AND status NOT IN ('complete','failed','dead_lettered','abandoned_confirmation','awaiting_confirmation')

- For each stale job: increment retry_count, if retry_count >= 3 set status='dead_lettered', else set status='queued' and clear worker_id and lease_expires_at

- Also check confirmation_expires_at: jobs where confirmation_expires_at < now() AND status='awaiting_confirmation' → set status='abandoned_confirmation'

- Print a summary of actions taken

- Supabase URL and service key from env SUPABASE_URL and SUPABASE_SERVICE_KEY

- Full error handling

```

---

### SCRIPT 3 — `preflight.py`

**Paste this into Claude CLI:**

```

I'm building GAIAKTA. Create a Python script at /mnt/b/projects/GAIAKTA/pipeline/preflight.py

Requirements:

- Python 3.12, stdlib only (uses subprocess to call ffprobe)

- Takes args: --video-path, --scan-id, --min-duration 30, --min-width 1920, --min-height 1080

- Runs ffprobe -v quiet -print_format json -show_streams -show_format on the video

*- Checks: codec must be h264 or hevc, duration >= min-duration seconds, video stream width*height >= min-width*min-height, file readable and non-zero size*

- On any failure: print specific message like "PREFLIGHT_FAIL reason=CODEC_INVALID codec=mpeg4" and exit code 1

- On pass: print "PREFLIGHT_OK duration={s}s resolution={w}x{h} codec={c}" and exit code 0

- Handle ffprobe not found, malformed JSON, missing streams gracefully

- Import and call log_lineage() and log_cost() stubs from ingest.py (or duplicate the stubs locally if import is complex)

```
```

SCRIPT 4 — `check_colmap.py`

Paste this into Claude CLI:

```

I'm building GAIAKTA. Create a Python script at /mnt/b/projects/GAIAKTA/pipeline/check_colmap.py

Requirements:

- Python 3.12, uses only stdlib + numpy

- Takes args: --colmap-output-dir (path to COLMAP sparse/0 folder), --total-frames (int), --scan-id

- Reads cameras.bin and images.bin and points3D.bin from the sparse reconstruction output using COLMAP binary format parsers (write the parsers — they are well documented binary formats)

- Computes these metrics:

  1. registered_ratio = registered_images / total_frames — must be >= 0.60

  2. point_count = number of 3D points — must be >= 10000

  3. mean_track_length = average observations per 3D point — must be >= 3.0

  4. mean_reprojection_error = average reprojection error across all points — must be <= 1.5px

  5. pose_spread = whether cameras are distributed around subject (compute convex hull of camera XZ positions, check hull area > threshold) — warn if poor but don't hard-fail

- If any hard metric fails: print "COLMAP_FAIL reason=X value=Y threshold=Z" and exit code 1
```

- If pose_spread warning: print "COLMAP_WARN pose_spread=poor" but continue

- On pass: print all 5 metrics as JSON and exit code 0

- Full error handling for missing or corrupt binary files

```

---

### SCRIPT 5 — `postprocess.py`

**Paste this into Claude CLI:**

```

I'm building GAIAKTA. Create a Python script at /mnt/b/projects/GAIAKTA/pipeline/postprocess.py

Requirements:

- Python 3.12, uses plyfile and numpy

- Takes args: --input-ply, --output-ply, --opacity-threshold 0.1, --scan-id

- Loads the .ply file using plyfile

- Reads the 'opacity' property from each Gaussian (it may be stored as raw logit — apply sigmoid if values are not in 0-1 range)

- Removes all Gaussians where sigmoid(opacity) < opacity-threshold

- Writes clean .ply to output path with same property structure as input

- Prints: "POSTPROCESS_OK input_gaussians={n} output_gaussians={m} removed={r} removed_pct={p:.1f}%"

- Sanity check: if output_gaussians < input_gaussians * 0.3, print "POSTPROCESS_WARN: removed more than 70% of Gaussians — check opacity threshold" but still write file and exit 0

- If output_gaussians < 10000, exit code 1 with "POSTPROCESS_FAIL: too few Gaussians remaining ({m}), scan may be unusable"

- Full error handling

```

---

### SCRIPT 6 — `check_quality.py`
```

Paste this into Claude CLI:

```

I'm building GAIAKTA. Create a Python script at /mnt/b/projects/GAIAKTA/pipeline/check_quality.py


Requirements:

- Python 3.12, stdlib + supabase-py

- Takes args: --nerfstudio-log (path to nerfstudio training log file), --colmap-metrics-json (path to JSON output from check_colmap.py), --scan-id

- Parses nerfstudio training log: find the final train_loss value (last epoch line containing "Train Loss:")

- Reads reprojection_error from colmap_metrics JSON

- Computes composite score:

  - Good: train_loss < 0.03 AND reprojection_error < 0.6

  - Fair: train_loss < 0.06 AND reprojection_error < 1.0

  - Poor: anything else

- For Fair: generate tip string based on which metric failed (e.g. "Try scanning in better lighting" for high loss, "Try scanning more slowly with more overlap" for high reprojection)

- Updates scan_jobs in Supabase: SET quality_score=X, quality_detail=JSON of all metrics WHERE id=scan_id

- Prints: "QUALITY_OK score=Good|Fair|Poor loss={x} reprojection={y}"

- Calls log_cost() stub with metric='quality_computed'

- Full error handling for missing or malformed log files

```

SCRIPT 7 — `finalize.py`

Paste this into Claude CLI:

```

I'm building GAIAKTA. Create a Python script at /mnt/b/projects/GAIAKTA/pipeline/finalize.py


Requirements:

- Python 3.12, uses boto3 (for R2), supabase-py, requests, hashlib

- Takes args: --splat-path (local clean splat.ply), --scan-id, --user-id, --raw-video-r2-key
```

- R2 credentials from env: R2_ENDPOINT, R2_ACCESS_KEY, R2_SECRET_KEY, R2_BUCKET

- Supabase from env: SUPABASE_URL, SUPABASE_SERVICE_KEY

- OneSignal from env: ONESIGNAL_APP_ID, ONESIGNAL_API_KEY

Implements 7 idempotent steps. Before each step, read scan_jobs.finalize_step — skip steps already completed:

STEP 1 — Upload splat.ply to R2:

- Compute local SHA-256 of splat.ply

- Upload to R2 at path: splats/{scan_id}/splat.ply

- Store ETag in scan_jobs.upload_etag, local hash in scan_jobs.file_hash (splat hash)

- Update finalize_step=1

STEP 2 — Verify upload integrity:

- Download splat from R2, compute SHA-256, compare to local hash

- If mismatch: exit code 1 "FINALIZE_FAIL step=2 reason=CHECKSUM_MISMATCH"

- Update finalize_step=2

STEP 3 — Write scan record to Supabase scans table:

- INSERT OR IGNORE into scans (id=scan_id, user_id, r2_path, status='pending_publish', created_at=now())

- Update finalize_step=3

STEP 4 — Validate CDN URL is live:

- Construct public CDN URL from R2 path

- HTTP HEAD request — must return 200

- Retry 3x with 10s backoff before failing

- Update finalize_step=4

STEP 5 — Mark job complete:

- UPDATE scan_jobs SET status='complete' WHERE id=scan_id

- UPDATE scans SET status='published' WHERE id=scan_id

- Update finalize_step=5

STEP 6 — Send OneSignal push notification:

- POST to OneSignal API: notify user_id "Your scan is ready to view on GAIAKTA"

- Include scan_id in notification data for deep-link

- Update finalize_step=6

STEP 7 — Delete raw video from R2:

- DELETE raw video object from R2 using --raw-video-r2-key

- Update finalize_step=7

After all steps: print "FINALIZE_COMPLETE scan_id={id}" and exit 0

Each step prints "FINALIZE_STEP_{n}_OK" when complete

If a step fails, print "FINALIZE_FAIL step={n} reason={r}" and exit 1 — next run will resume from that step

```

---

### SCRIPT 8 — `segment.py`

**Paste this into Claude CLI separately — this one needs SAM2 installed first:**

```

I'm building GAIAKTA. Create a Python script at /mnt/b/projects/GAIAKTA/pipeline/segment.py

Requirements:

- Python 3.12, uses SAM2 (sam2 pip package), PIL, numpy, supabase-py

- Takes args: --frames-dir (folder of extracted 4K JPEGs), --output-dir (where to write masked frames), --scan-id, --num-keyframes 7, --confidence-threshold 0.75

- SAM2 model checkpoint path from env: SAM2_CHECKPOINT (default: sam2_hiera_large.pt)

Step 1 — Select keyframes:

  - List all JPEGs in frames-dir, sort by name

  - Pick num-keyframes evenly spaced (first, last, and evenly distributed middle frames)

Step 2 — SAM2 automatic mask generation on each keyframe:

- Use SAM2AutomaticMaskGenerator with points_per_side=32

*- For each keyframe, select the mask with highest predicted_iou that covers the most central area of the frame (score = iou * centrality where centrality = 1 - distance_of_mask_centroid_from_frame_center / max_possible_distance)*

- Store per-keyframe confidence scores

Step 3 — Compute overall confidence:

- Mean of per-keyframe confidence scores

- Update scan_jobs SET mask_confidence=X WHERE id=scan_id

- If mean confidence < confidence-threshold: print "SEGMENT_WARN low_confidence={score:.3f} — multi-frame preview flagged"

Step 4 — Generate preview images:

- For 3 keyframes (first, middle, last): create side-by-side composite: original frame | masked frame (background blacked out)

- Save as preview_1.jpg, preview_2.jpg, preview_3.jpg in output-dir

- These are shown to user for confirmation

Step 5 — Set confirmation TTL:

- UPDATE scan_jobs SET status='awaiting_confirmation', confirmation_expires_at=now()+24h WHERE id=scan_id

- Print preview paths and "SEGMENT_OK confidence={score:.3f} awaiting_user_confirmation"

- EXIT here — pipeline pauses. Resume is triggered by user confirmation webhook.

Step 6 — Mask propagation (called separately with --propagate flag + --confirmed-keyframe-index N):

- Load the confirmed keyframe's mask

- Use SAM2VideoPredictor to propagate mask through all frames

- Apply mask: set background pixels to black in each frame

- Write masked frames to output-dir

- Print "PROPAGATION_OK frames_masked={n}"

```

---
```

SCRIPT 9 — `pipeline.py`

Paste this into Claude CLI last, after all above scripts are tested:

```

I'm building GAIAKTA. Create a Python script at /mnt/b/projects/GAIAKTA/pipeline/pipeline.py

This is the master orchestrator. It calls all stage scripts in order, manages state transitions in Supabase, and starts/stops the heartbeat thread.

Requirements:

- Python 3.12, uses subprocess, threading, supabase-py

- Takes args: --video-path, --scan-id, --user-id, --raw-video-r2-key, --colmap-output-dir, --frames-dir, --masked-frames-dir, --splat-output-dir

- All env vars delegated to child scripts (SUPABASE_URL, SUPABASE_SERVICE_KEY, R2_*, SAM2_CHECKPOINT, ONESIGNAL_*)

Flow:

1. Import heartbeat.py start_heartbeat() — start heartbeat thread immediately

2. Run ingest.py as subprocess — on non-zero exit, update status=failed with reason, stop heartbeat, exit

3. Run preflight.py — same failure handling

4. UPDATE scan_jobs SET status='extracting_frames'

5. Run ffmpeg command for frame extraction (4K, 2-3 FPS) — hardcoded command with configurable fps arg

6. UPDATE scan_jobs SET status='segmenting'

7. Run segment.py — exits after setting awaiting_confirmation. Pipeline suspends here.

8. Pipeline prints "PIPELINE_SUSPENDED awaiting_user_confirmation scan_id={id}" and exits 0

   (Resume is a separate invocation with --resume-from confirmed)

RESUME MODE (--resume-from confirmed --confirmed-keyframe-index N):

9. Run segment.py --propagate --confirmed-keyframe-index N

10. UPDATE scan_jobs SET status='reconstructing'

11. Run COLMAP commands (feature_extractor, exhaustive_matcher, mapper) — use working flags from GAIAKTA first run

12. Run check_colmap.py — on fail, update status=failed, stop heartbeat, exit

13. UPDATE scan_jobs SET status='training'
```

14. Run ffmpeg downscale 4K→1080p

15. Run ns-train splatfacto with --downscale-factor 1 and all working flags from first run

(heartbeat keeps pinging during long training)

16. UPDATE scan_jobs SET status='exporting'

17. Run ns-export gaussian-splat

18. UPDATE scan_jobs SET status='postprocessing'

19. Run postprocess.py — on fail, update status=failed

20. UPDATE scan_jobs SET status='quality_check'

21. Run check_quality.py

22. Call log_lineage() stub — log splat artifact

23. Call log_cost() stub — log gpu_seconds estimate

24. UPDATE scan_jobs SET status='finalizing'

25. Run finalize.py

26. Stop heartbeat thread

27. Print "PIPELINE_COMPLETE scan_id={id}"

Error handling: any stage failure → UPDATE status=failed, reason=<stage name + error>, stop heartbeat, exit 1

All stage transitions print timestamped progress: "[HH:MM:SS] STAGE_N starting..."